

Building a code reviewer with Google ADK

Six phases, one layer at a time: from a bare model to a reviewer whose verdict is decided by code.

THE PLAN

The next hour

0:00	Welcome and ADK core concepts	20 min
0:20	Phase 1 · Hello world	5 min
0:25	Phase 2 · Preflight	5 min
0:30	Phase 3 · Changed files	5 min
0:35	Phase 4 · First review	8 min
0:43	Phase 5 · Parallel lanes	7 min
0:50	Phase 6 · The reviewer	10 min
1:00	Going further: phase 7	3 min
1:03	Q&A	

After this session you can

- build an agent with tools and a guardrail
- wire agents into a workflow with typed answers
- move decisions out of the model and into code

Phases 1 to 6 live. Phase 7 is in the repository for self-study.

WHERE WE'RE GOING · LIVE

One command, one verdict a build can trust

```
$ uv run python -m phase_6_reviewer.review --head feature/payments
```

The key, found by code

Two blockers from the secrets gate, each with its exact file and line.

Findings that quote the diff

Three model lanes judge; a finding the diff does not bear out is dropped.

An exit code

1 here: a blocker. The report ends with what the run cost.

Over the next hour we build this, one phase at a time.

WHAT WE BUILD

A reviewer for one git branch

IN

A repository, a base branch and the branch under review.

OUT

Typed findings, a verdict, and an exit code a build can act on.

RUNS

In adk web from a chat message, or from the shell as one command.

`feature/payments` · 4 planted defects

- `subprocess.run(..., shell=True)`
- a hard-coded AWS key pair (Amazon's documented example values)
- a function nothing tests
- a copy-pasted block

Built by `scripts/make_demo_repo.py`: predictable findings, no network.

ADK: agents as ordinary Python

Google's open-source Agent Development Kit. You write agents, tools and the graph that connects them in Python, and ADK runs them.

Any model

GitHub Copilot through LiteLLM, Gemini natively. Here: both, one setting apart.

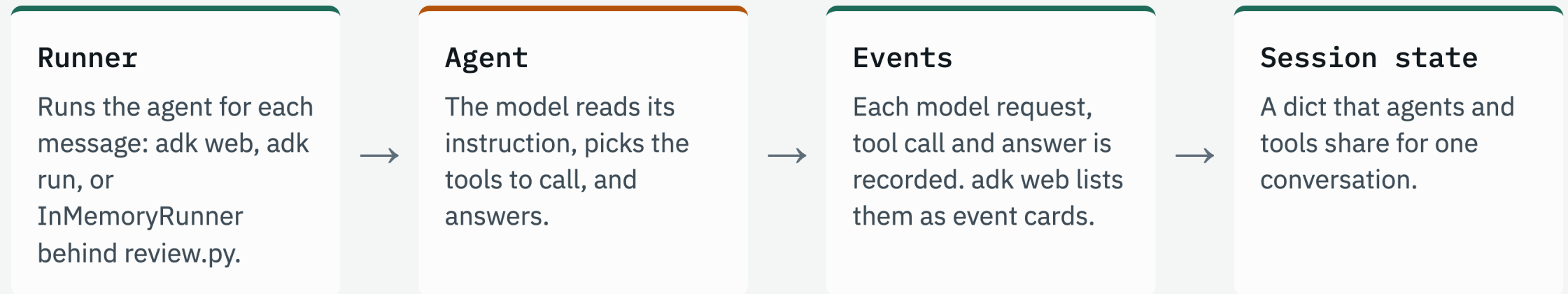
Tools to watch it run

`adk web` opens a browser UI that lists every event; `adk run` is a terminal chat.

Code where you want control

Workflows, callbacks and plugins let code decide what a model should not.

Every step of a turn becomes an event



State is how agents and tools pass work on

`output_key="diff"`

saves an agent's answer to state (from phase 2)

`{diff}`

reads it into the next agent's instruction (phase 4)

`tool_context.state`

lets a tool read and write it (phase 3)

`temp:`

a key with this prefix lasts one turn (phase 4)

Hooks let code step in around every step

Hook	Runs	In the demo
<code>before_agent_callback</code>	before an agent starts	phase 4: skips the reviewer without evidence
<code>before_model_callback</code>	before each model request	phase 6: redacts secrets, counts the call
<code>after_model_callback</code>	after each model answer	phase 6: counts tokens for the spend line
<code>on_model_error_callback</code>	when the provider fails	phase 4: a sentence instead of a traceback
<code>before_tool_callback</code>	before each tool call	phase 3: the path guardrail
<code>after_tool_callback</code>	after each tool result	phase 6: redacts what tools return

Return nothing and the step runs. Return a value and it replaces the step.

A callback belongs to one agent. A plugin puts the same hooks on the whole App.

Each phase adds one ADK idea

Phase	New ADK idea	Answers
1 · Hello world	Agent, root_agent, adk web and adk run	one agent, two providers
2 · Preflight	a function tool, the {status} envelope	“Do this repo and branch exist?”
3 · Changed files	ToolContext.state, before_tool_callback	“What did the branch change?”
4 · First review	Workflow, output_key → {diff}, output_schema	“Is this change OK?”
5 · Parallel lanes	fan-out, JoinNode, max_concurrency	three opinions at once
6 · The reviewer	App and plugins, the verdict in code	an exit code a build can trust
7 · Hardened	layers, retries, gates, rules	self-study

One phase's lesson, exactly: `git diff --no-index -- phase_3_changed_files phase_4_first_review`

THE THREAD THROUGH EVERY PHASE

Models judge. Code decides.

We start with a model doing everything. Each phase moves one more decision into code, and that is what makes the agent deterministic and controllable.

A single agent file or an ADK workflow?

Dimension	Single agent file the prompt defines the steps and flow	ADK workflow code defines the steps and flow
Development speed	✓ Very fast to build and change ✗ Can get messy as it grows	✗ More work up front ✓ Easier to grow cleanly over time
Determinism	✗ The agent may skip, merge or reinterpret steps	✓ Steps run exactly as coded
Reliability	✗ Depends heavily on model behaviour	✓ More predictable and consistent ✓ A failed step is reported, never approved
Control	✗ Limited control over what runs, and when	✓ Full control over order, retries and permissions
Observability	✗ Hard to know which steps really ran	✓ Every step is logged and traced
Debugging	✗ Hard to find where it went wrong	✓ Easy to find the step that failed
Testing	✗ Mostly tested by checking the final answer	✓ Each step and the whole flow can be tested ✓ Tests run offline, with a fake model

When it grows, and when it matters

Dimension	Single agent file the prompt defines the steps and flow	ADK workflow code defines the steps and flow
Adaptability	✓ Can change its approach when needed	✗ Only adapts if explicitly coded ✓ The model can be swapped with one setting
Maintenance	✓ Easy at first ✗ Harder as the prompt grows large	✓ Easier to maintain as the system grows
Governance & compliance	✗ Hard to enforce rules and guarantees	✓ Audit, security and approval rules enforced
Failure recovery	✗ May start over from scratch after a failure	✓ Can retry, or resume from where it stopped
Scalability	✗ Large workflows become hard to manage	✓ Handles large, complex workflows better ✓ Independent steps run in parallel
Business-critical processes	✗ Risky where a process must be strict	✓ The safer choice for strict processes
Research / creative tasks	✓ Usually a good fit	✗ Can feel over-engineered

Where ADK makes the difference

Situation	Single agent file	ADK workflow
A secret in the code	✗ Sent to the model, printed in the answer	✓ Hidden from the model, flagged as a blocker
A file too long to read in full	✗ Approved anyway, with no warning	✓ Names the file and refuses to approve
A build needs pass or fail	✗ A sentence to read and trust	✓ An exit code the pipeline can act on
A spending limit	✗ Nothing stops it from spending	✓ Stops at the limit and says so
Testing the flow	✗ Needs the live model	✓ Runs offline, with a fake model

Same model, same tools. The difference is what the code guarantees.

Code runs the process, the model judges

Control of the whole process

Code sets the order, enforces the rules and decides the verdict, through workflows, callbacks and plugins.

Deterministic, repeatable runs

The same steps in the same order on every run, and a verdict computed in Python: exit 0, 1, 2 or 3.

Every step observable

Every model call, tool call and answer is an event, with tokens per agent and OpenTelemetry traces.

Intelligence exactly where it's needed

The model reasons only in the steps that need judgement: here, three lanes reading the diff.

A single agent file is still the faster start when a person reads every answer. And it is not either-or: an ADK agent can load and use `SKILL.md` skills.

One agent, two providers, one switch

```
# agent.py
root_agent = Agent(
    model=build_model(),
    name="phase_1_hello_world",
    instruction="... Greet ...",
)
```

```
# config.py, the only file that names a provider
provider = (os.environ.get("REVIEW_PROVIDER", "")
            .strip().lower() or "copilot")

if provider == "gemini":
    return model # ADK speaks Gemini natively
return LiteLlm(model=f"github_copilot/{model}")
```

The folder is the agent, and `root_agent` is what the loader looks for. No other file knows which provider runs.

Abridged. No tools, no state, no callbacks yet.

DEMO

```
uv run adk run phase_1_hello_world
uv run adk web . → hello
REVIEW_PROVIDER=gemini, restart, ask again
```

A tool's docstring is its schema

```
+ def inspect_repository(repo: str, base: str, head: str) -> dict:
+     """Confirm repo is a git repository, that base and head
+     exist, and find their merge base."""
+     return {"status": "success", "head_sha": ..., ...}

root_agent = Agent(model=build_model(), name="phase_2_preflight",
+                 tools=[inspect_repository], output_key="preflight")
```

A tool answers {status: ...} and never raises at the model: an envelope lets the model explain.

Abridged. Lines marked + are new in this phase.

DEMO

```
review <demo repo>, branch
feature/payments against main
```

The guardrail runs before the tool

```
+ def only_changed_paths(tool, args, tool_context) -> dict | None:
+     if tool.name != "show_diff":
+         return None                                # lets the call through
+     listed = tool_context.state.get("review_files")
+     if ... or path not in listed:
+         return {"status": "error", ...}            # answers the model; the tool never runs
+     return None
+ root_agent = Agent(..., before_tool_callback=[only_changed_paths])
```

One tool's answer is the next tool's boundary: `changed_files` writes the allowlist, `show_diff` is held to it.

Abridged from `callbacks.py` and `agent.py`. Lines marked + are new in this phase.

DEMO

the same message, then
the diff of `src/payments/charge.py`
the diff of `/etc/passwd`

State is the wire between two agents

```
collector = LlmAgent(tools=[inspect_repository, changed_files, show_diff],
+                   output_key="diff")
+ reviewer = LlmAgent(instruction="Review the change below ... {diff}",
+                   output_schema=Review, output_key="review")
+ root_agent = Workflow(edges=[("START", collector, reviewer)])
```

Gathering and judging are separate agents: the collector has the tools, the reviewer has the schema.

Abridged from agent.py. Lines marked + are new in this phase. The command's exit 0 means judged, not clean: phase 4 never exits 1.

DEMO · THE SAME GRAPH AS A COMMAND

```
uv run python -m
phase_4_first_review.review
--head feature/payments
```

Findings are typed, failures are answers

```
class Finding(BaseModel):
    file: str
    line: int | None
    severity: Literal["blocker", "major", "minor"]
    title: str
    evidence: str      # the diff lines, verbatim
    suggestion: str

class Review(BaseModel):
    findings: list[Finding]
    summary: str
```

schemas.py, abridged. ADK puts the parsed object in state, not text.

on_model_error_callback

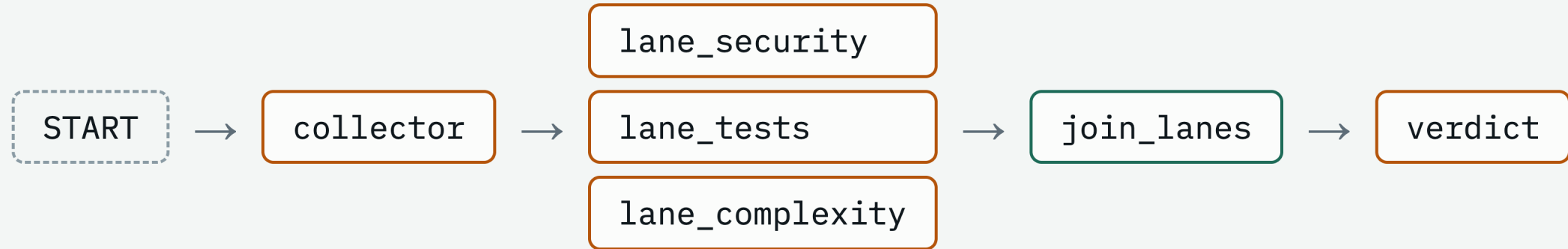
A provider error becomes the agent's answer: one sentence, never a traceback.

before_agent_callback

With nothing gathered, the reviewer skips itself and says why.

“Did not look” must never read as “found nothing”.

One edge fans out to three lanes



 an agent that calls a model  JoinNode: code that waits for every branch

```
lanes = tuple(build_lane(*lane) for lane in LANES)      # a lane is a row: name, role, budget
root_agent = Workflow(edges=[("START", collector, lanes, join, verdict)], max_concurrency=3)
```

DEMO • THE SAME MESSAGE IN ADK WEB, THEN THE COMMAND

```
uv run python -m phase_5_parallel_lanes.review --head feature/payments
```

Where phase 5 still trusts the model

Step	Phase 5: a model	Phase 6: code
Gathering the diff	tool calls chosen by a model	Python, before any model runs
Checking each quote	nothing checks it	whole lines of its own file's diff, or dropped
Line numbers	the model's guess	read off the diff
Secrets in the diff	sent to every lane, and printed back	redacted for models, a blocker from a gate
The verdict	a model writes APPROVED	a function decides, with an exit code
A lane that fails	the verdict agent is asked to say so	named, and the run can never approve

Phase 5 spends model requests just to gather the diff. Phase 6 gathers it with git and spends three requests, one per lane.

Only one of five steps calls a model

1 • code

Evidence

Read-only git resolves both branches, lists the changed files, reads every diff.

2 • code

Gate

Every added line is checked against 13 kinds of credential. A hit is a blocker.

3 • models

Judgement

Three lanes judge the same diff, as data, never as instructions.

4 • code

Verdict

A quote must be whole lines of its file's diff, or it is dropped. Duplicates folded.

5 • code

Report

What was found, dropped, folded, redacted and not read, and what it cost.

Everything that decides the exit code is Python.

This is the command. In adk web chat the lanes and plugins run too, but the verdict is still a model's.

Plugins carry policy no agent can skip

```
def build_app(*, chat: bool) -> App:
    plugins = [RedactSecretsPlugin(), UsageLedger(), ReportProviderErrors()]
    return App(name=NAME, root_agent=build_workflow(chat=chat), plugins=plugins)

app = build_app(chat=True)      # the command builds it with chat=False: lanes only
```

RedactSecretsPlugin

Scrubs credential shapes from every request before a model sees it.

UsageLedger

Counts every call's tokens; the report ends with what the run cost.

ReportProviderErrors

A failed lane is named with its reason; the others finish.

The verdict is a function, not a prompt

```
def decide(outcome: Outcome, fail_on: str) -> tuple[str, int]:
    if outcome.errors or outcome.unread or _ungrounded_at(outcome, fail_on):
        return "DEGRADED", EXIT_DEGRADED
    worst = max((RANK[item.finding.severity] for item in outcome.kept), default=0)
    return ("REQUEST_CHANGES", EXIT_FINDINGS) if worst >= RANK[fail_on]
        else ("APPROVED", EXIT_CLEAN)
```

- 0 Approved.
- 1 A finding at the bar: blocker by default, --fail-on to change it.
- 2 A usage or readiness error before any model call, or nothing to review.
- 3 A hole: a failed lane, an unread file, or a finding at the bar the diff does not bear out.

A hole never approves.

A function, so it is tested like one: the suite runs every phase offline, on a fake model.

Two fixes, and the build can act

Step 1 · exit 1

The run from the start

The hard-coded key, found by the code gate, and the shell call, found by a lane.

Step 2

Commit the two fixes

```
uv run python  
scripts/make_demo_repo.py -  
-fix
```

No shell=True, and keys read from the environment.

Step 3 · exit 0

Review again

the same command

What is left sits below the bar: findings for a person, not a failed build.

Then read the end of the report: what the run cost.

Phase 7 hardens the same reviewer

core/

the domain: no
process, no model

rules/

a project's own rules

collect/

evidence: git and the
ruff gate

judge/

the model boundary:
lanes, verify

deliver/

the report

Each group imports only the groups to its left, and a test fails any import that points back.

What it adds to phase 6

- retries on both providers, and an optional fallback model
- a token ceiling that reserves before it spends
- a ruff gate: a second check by code
- --verify: a second opinion on lane findings
- each file's blast radius, from the import graph
- a project's own rules, chosen by profile

```
uv run pytest -q tests/test_layering.py
uv run python -m phase_7_hardenened.review \
    --head feature/payments --verify
# then the same with --max-tokens 2000,
# and with --profile gates-only
```

A rule is a class, a profile a list

```
# rules/no_print.py, abridged
class NoPrint(Rule):
    id = "no-print"
    severity = "minor"
    title = "print() left in library code"
    fix = "Use the logger."

    def check(self, path: str, line: str) -> bool:
        ... # a print token, then "("
```

```
# config.toml, under [profiles], shortened
gates-only = ["secrets", "lint",
              "no-print", "money-in-cents", ...]
```

- Drop a file in rules/ and it runs, on every line the branch added, or on the tree at the head.
- A profile lists check ids: your rules plus secrets, lint and the three lanes.
- No --profile runs every check. gates-only runs no lane: no model, no key.
- A rule that will not load, or a broken profile or config, stops the run before it starts (exit 2).
- Nothing is read from the repository under review.

TAKEAWAYS

Four rules to take home

Grow an agent one layer at a time

Each phase is a working folder, and the diff between two folders is one lesson.

Treat what the model picks as untrusted

Tools answer with envelopes, guardrails sit in front of them, every string is capped.

Give the shape to code

Workflow edges, JoinNode and plugins decide the flow. Prompts only fill it in.

Decide in code, name every hole

Models judge, code decides, and “did not look” never reads as “found nothing”.

Questions?

Repository: [repository link]

The recording and its chapter times will be in the meeting chat.

TRY IT YOURSELF

1 · Install

`uv sync`, then copy `.env.example` to `.env`

2 · Log in to Copilot, once

`uv run python scripts/copilot_login.py`

3 · Build the demo repository

`uv run python scripts/make_demo_repo.py`

4 · Open every phase in the browser

`uv run adk web .`

Feedback: [feedback form link]

APPENDIX

Backup: what each demo shows

One slide per demo, captured at rehearsal. Jump here when a live call is slow or fails, then carry on.

Hello world

```
$ uv run python scripts/ask.py phase_1_hello_world hello  
[tokens] in=106 out=45  
Hello! To review a git branch, I would need access to the  
repository and the name of the branch you'd like reviewed.  
I don't have any tools or access yet, so I can't start the  
review process at this time.
```

Expected

A one-sentence greeting, what a review would need, and that it has no tools yet. Click the answer's event card: the side panel shows the request and its tokens.

phase_1_hello_world → hello

The preflight block

```
$ uv run python scripts/ask.py phase_2_preflight \  
    "review <tmp>/adk-demo-repo, branch feature/payments against main"  
[tokens] in=1114 out=124  
repository    <tmp>/adk-demo-repo  
branch        feature/payments @ 840818199  
against       main @ 10559fe72  
merge-base    10559fe72  
ahead         1 commit(s)
```

Expected

Five lines: repository, branch @ sha, against @ sha, merge-base, commits ahead.

branch feature/nope against main

The tool's own error, repeated in one sentence:

head branch 'feature/nope' does not exist in <tmp>/adk-demo-repo

Changed files, and a refusal

```
$ uv run python scripts/ask.py phase_3_changed_files "review ..."  
[tokens] in=2637 out=241  
Repository: <tmp>/adk-demo-repo  
Branch under review: feature/payments @ 84081819974f...  
Against base: main @ 10559fe7217f...  
Merge-base: 10559fe7217f...  
Commits ahead: 1  
  
Changed files:  
- M src/payments/charge.py  
- A src/payments/config.py  
- A src/payments/refund.py  
- A src/payments/report.py
```

Expected

The preflight lines and four files:
charge.py modified, config.py,
refund.py and report.py added.

show me the diff of /etc/passwd

The guardrail answers; the tool
never runs:

'/etc/passwd' is not one of the
files this change touched; choose
one changed_files listed

The first typed review

```
$ uv run python -m phase_4_first_review.review --head feature/payments
{
  "findings": [
    {
      "file": "src/payments/charge.py",
      "line": 11,
      "severity": "blocker",
      "title": "Command injection ... unsanitized printer name",
      ...
    },
    { "file": "src/payments/config.py", "severity": "blocker", ... },
    { "file": "src/payments/refund.py", "severity": "major", ... },
    { "file": "src/payments/report.py", "severity": "major", ... }
  ],
  "summary": "... critical security issues (command injection, ...)"
}
```

Expected

The review as JSON: findings with file, severity, quoted evidence and a fix, one of them the shell=True line in charge.py. Exit 0: judged.

```
uv run python -m
phase_4_first_review.review --head
feature/payments
```

Three lanes side by side

```
$ uv run python -m phase_5_parallel_lanes.review \  
  --head feature/payments  
REQUEST CHANGES  
  
### security  
#### src/payments/charge.py (blocker)  
**Command injection vulnerability via unsanitized printer name**  
#### src/payments/config.py (blocker)  
**Hardcoded AWS credentials in source code**  
### tests  
#### src/payments/refund.py (major)  
#### src/payments/charge.py (major)  
#### src/payments/report.py (major)  
### complexity  
#### src/payments/charge.py (blocker)  
#### src/payments/config.py (blocker)
```

Expected

Three lane calls side by side in the Traces view, the join once after all three, then a verdict with findings grouped by lane: REQUEST CHANGES.

phase_5_parallel_lanes, same message

Before and after the two fixes

The fixes: `charge.py` no longer uses `shell=True`, and `config.py` reads the AWS keys from the environment.

Before: exit 1, the key found by the gate

```
$ uv run python scripts/make_demo_repo.py
$ uv run python -m phase_6_reviewer.review \
  --head feature/payments
### Verdict: REQUEST_CHANGES
### gate_secrets
- blocker hard-coded credential
  (aws_access_key_id) (config.py:4)
- blocker hard-coded credential
  (aws_secret_access_key) (config.py:5)
### lane_security
- blocker Command injection ... (charge.py:22)
### lane_tests
- major No tests for refund function
- major No tests for daily_total and format_total
- secrets redacted before any model call: 6
- spend: 3 model call(s) completed of 3
  attempted, 4,350 tokens
```

After the two fixes: exit 0

```
$ uv run python scripts/make_demo_repo.py --fix
$ uv run python -m phase_6_reviewer.review \
  --head feature/payments
### Verdict: APPROVED
### gate_secrets
- no findings
### lane_security
- major No validation for negative refund amounts
- minor Sensitive credentials ... environment
### lane_tests
- major No tests for daily_total function
- major No test for receipt function
### lane_complexity
- no findings
- secrets redacted before any model call: 0
- spend: 3 model call(s) completed of 3
  attempted, 4,182 tokens
```